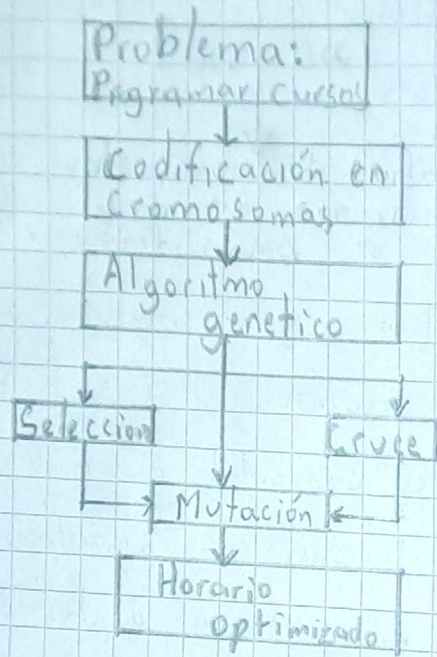


Racingur Prada Olaya 2899747

Artículo 1: Revisión de Algoritmos Genéticos Aplicados al Problema de la Programación de cursos Universitarios (2011)

Este trabajo presenta un análisis sistemático del uso de algoritmos genéticos en la programación de cursos en universidades. La programación en cursos implica asignar materias, aulas y horarios de forma que se optimicen recursos y se satisfagan restricciones (disponibilidad de docentes, capacidad de aulas, etc.). El artículo revisa diversas implementaciones de algoritmos genéticos, comparando variaciones en la representación de las soluciones, operadores de cruce y mutación, así como criterios de selección. Se analizan métricas de desempeño como tiempo de cómputo y grado de cumplimiento de las restricciones. Entre las conclusiones, se destaca que los algoritmos genéticos muestran gran flexibilidad para adaptarse a escenarios complejos y cambiantes, aunque su efectividad depende mucho del ajuste de parámetros y la combinación con otras técnicas (heurística o híbridas). El autor sugiere líneas futuras como la integración de preferencias de los usuarios y la incorporación de factores externos como cambios de infraestructura. En suma, el estudio proporciona un mapa del estado del arte hasta 2011 y argumenta que la programación de cursos seguirá siendo un terreno fértil para la optimización evolutiva.

gráfica:

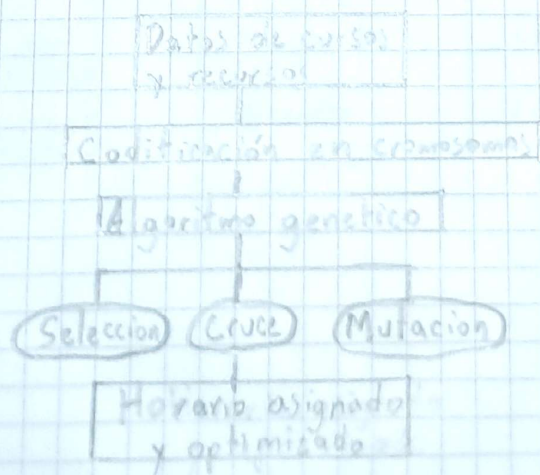


Reflexión: Esta revisión evidencia como los algoritmos genéticos, inspirados en la evolución natural, pueden resolver problemas complejos de asignación de recursos. Mas allá de su aplicación puntual en horarios universitarios, plantea un marco adaptable a cualquier escenario donde la optimización y las restricciones múltiples convivan. Nos recuerda que la clave no es solo el algoritmo, sino cómo se modela el problema.

Artículo 2: Implementación de un algoritmo genético para la gestión y asignación de horarios para los colegios oficiales (2018)

Este trabajo describe el diseño e implementación de un sistema para organizar y asignar horarios en instituciones educativas públicas, utilizando algoritmos genéticos como técnica principal de optimización. El problema que aborda es complejo debido a las múltiples restricciones: disponibilidad de docentes, número y tipo de aulas, carga horaria de cada asignatura y criterios institucionales. El autor detalla la codificación genética de las posibles soluciones, la función de aptitud utilizada para evaluar la calidad de los horarios y la configuración de operadores de cruce y mutación para mejorar combinaciones generadas. Se enfatiza el uso de datos reales extraídos de colegios oficiales y se mide el desempeño del algoritmo en términos de reducción de conflictos y mejoras en la distribución de clases. Entre las conclusiones, se señala que el método logró optimizar la asignación de recursos y cumplir con las restricciones en un alto porcentaje, reduciendo la intervención manual. El artículo propone como trabajo futuro integrar módulos de interfaz más amigables y extender el sistema a escenarios de planificación más amplios.

Grafica:

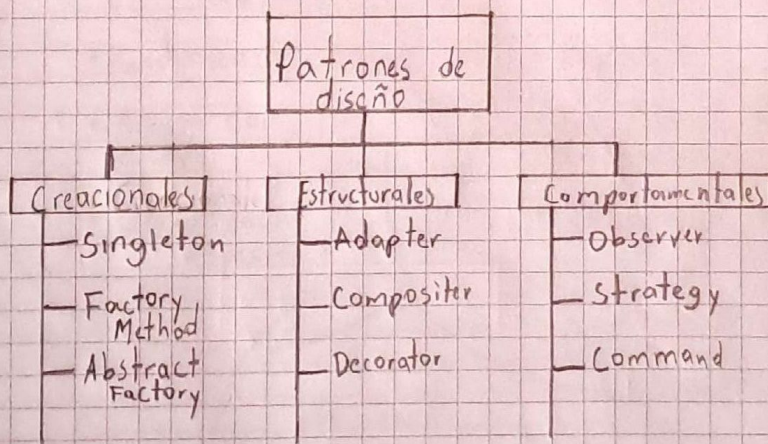


Reflexión: Este trabajo traslada la teoría a un contexto real y socialmente relevante: la educación pública. La reflexión aquí es que la tecnología no solo optimiza procesos, sino que puede democratizar el acceso a soluciones eficientes, reduciendo carga administrativa y mejorando la experiencia de estudiantes y docentes.

Artículo 3: Patrones de diseño orientado a objetos (2009)

Este texto funciona como una guía conceptual y técnica para aplicar patrones de diseño en el desarrollo de software orientado a objetos. Comienza explorando la importancia de los patrones como soluciones probadas a problemas recurrentes de arquitectura y diseño, fomentando la reutilización y la claridad del código. El artículo revisa patrones clásicos como Singleton, Observer, Factory Method, Adapter y distintos lenguajes. Se profundiza en cómo los patrones permiten desarrollar componentes, facilitar la mantenibilidad del software y mejorar la estabilidad de sistemas complejos. Además, se discuten errores comunes al aplicarlos, como el uso innecesario de patrones que aumentan la complejidad. El autor plantea que conocer y dominar los patrones no significa aplicarlos indiscriminadamente, sino entender su propósito y cuándo aportan valor real. La obra cierra con recomendaciones para integrarlos en procesos de desarrollo ágil y como material formativo en carreras de ingeniería de software.

Grafico:

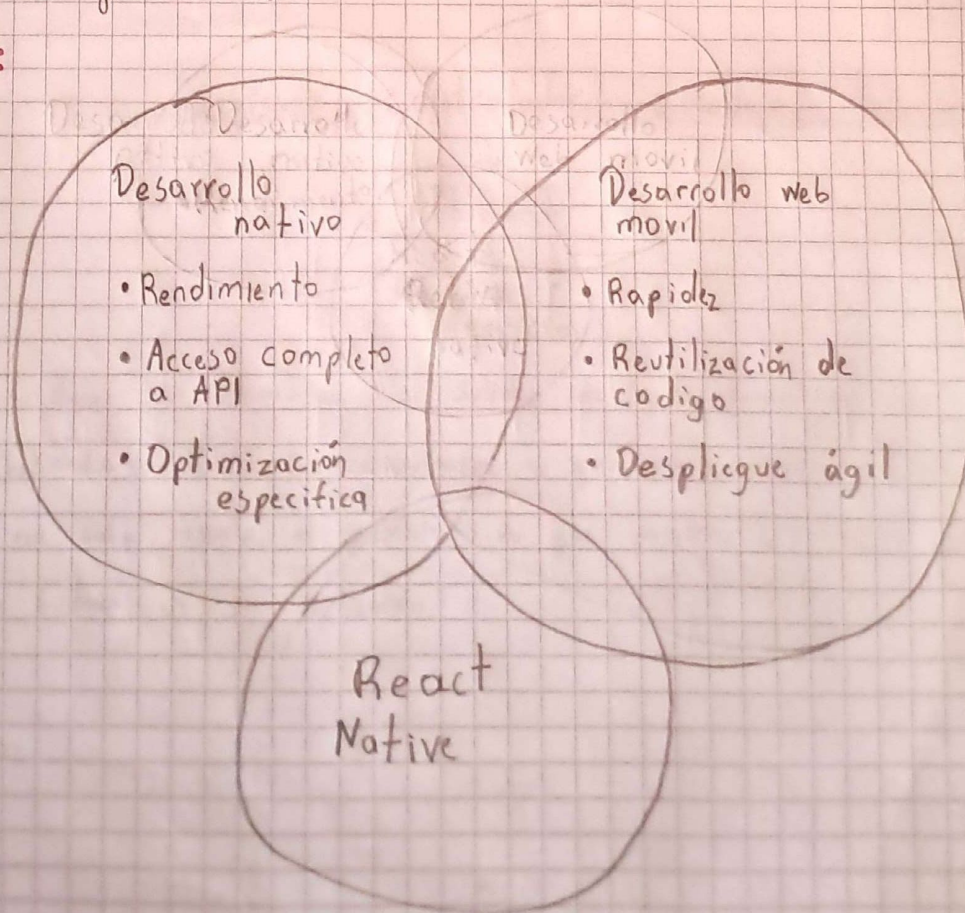


Reflexión: Aunque los patrones de diseño son un concepto maduro, su vigencia radica en que encapsulan buenas prácticas que trascienden lenguajes y modas tecnológicas. Este artículo nos recuerda que la arquitectura de software es tanto un arte como una ingeniería, y que la reutilización inteligente es una forma de sostenibilidad en el desarrollo.

Artículo 4: React Native: acortando las distancias entre desarrollo y diseño móvil multiplataforma (2019)

Este artículo analiza cómo React Native permite crear aplicaciones móviles nativas para iOS y Android usando un único código base en JavaScript, manteniendo un rendimiento cercano al de las apps desarrolladas de forma nativa. Explica la arquitectura del framework, su puente ("bridge") que comunica el código JavaScript con componentes nativos, y como esto favorece la reutilización de lógica y diseño entre plataformas. Se abordan casos de uso en proyectos reales, ventajas como la necesidad de conocimiento nativo para integrar ciertas APIs. También se destacan buenas prácticas para optimizar rendimiento y experiencia de usuario, así como la creciente comunidad y ecosistema de librerías que amplía sus posibilidades.

Gráfica:



Reflexión: Este artículo muestra cómo React Native rompe la barrera entre el desarrollo nativo y el multiplataforma, permitiendo que un solo código llegue a múltiples dispositivos. La reflexión es que la convergencia entre diseño y desarrollo no solo ahorra tiempo, sino que redefine la colaboración entre equipos técnicos y creativos.

Artículo 5: Implementación de un framework para el desarrollo de aplicaciones web utilizando patrones de diseño y arquitectura MVC/REST (2010)

Este trabajo presenta la creación de un framework propio para aplicaciones web que combine la arquitectura MVC (Modelo-Vista-Controlador) con servicios REST. Describe el proceso de diseño, la implementación de cada capa y la forma en que MVC ayuda a separar responsabilidades, facilitando el mantenimiento y escalabilidad del software. El autor explica cómo los patrones de diseño utilizados optimizan la gestión de datos y mejoran la interacción cliente-servidor, además de incluir ejemplos prácticos de rutas y controladores. Entre sus conclusiones, resalta que la combinación MVC/REST puede aplicarse eficazmente a diversos contextos, mejorando la organización del código y la interoperabilidad de sistemas.

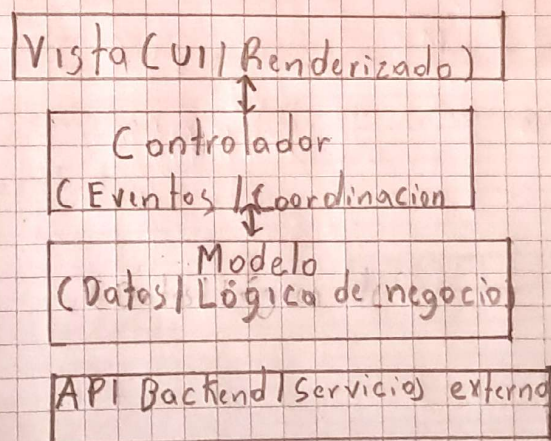
Grafica:



Reflexión: Aquí se combina la solidez de los patrones de diseño con la flexibilidad de REST. La enseñanza es que una arquitectura bien pensada no solo resuelve el presente, sino que prepara el terreno para la evolución futura de la aplicación.

Artículo 6: Modelo de adecuación y arquitectura de aplicaciones utilizando patrones arquitectónicos Modelo Vista Controlador en el Frontend (2015) Este artículo propone un modelo de adecuación para decidir cuándo y cómo aplicar el patrón MVC en el desarrollo de interfaces de usuario. Analiza problemas habituales en el frontend como la alta dependencia entre componentes y la dificultad para escalar proyectos, y muestra cómo MVC, bien implementado, puede mejorar la modularidad y reducir la complejidad. Se detallan adaptaciones del patrón para entornos web modernos, integrando frameworks JavaScript, y se ofrecen métricas de evaluación para medir la calidad de la arquitectura. El trabajo concluye que la correcta aplicación de MVC en el frontend facilita el mantenimiento, permite la reutilización de componentes y mejora la experiencia del usuario final.

Gráfica:



Reflexión: Este trabajo recuerda que el MVC no es solo para el backend: aplicado al frontend, aporta orden y escalabilidad. La reflexión es que la claridad estructural en la interfaz de usuario es tan importante como en la lógica de negocio.

Artículo 93 Principios de diseño de APIs REST (2013)

Este artículo profundiza en los pilares esenciales para construir APIs REST que son claras, escalables y fáciles de mantener. Comienza definiendo el concepto de recurso como elemento central en la arquitectura, destacando la importancia de identificarlos de forma única mediante URIs claras, predecibles y jerárquicas. Se enfatiza el uso correcto de los métodos HTTP —GET, POST, PUT, DELETE— respetando su semántica y asegurando que cada operación se asocie al verbo adecuado. El autor subraya la relevancia de emplear códigos de estado HTTP precisos que transmitan de manera inequívoca el resultado de las solicitudes, mejorando así la comunicación entre cliente y servidor. Otro punto central es el manejo consistente de errores, que debe incluir mensajes comprensibles para desarrolladores y sistemas automáticos. También se analiza la conveniencia de elegir formatos de intercambio de datos comunes, como JSON o XML, para favorecer la interoperabilidad, así como la estrategia de versionado que permite evolucionar la API sin interrumpir la compatibilidad con implementaciones existentes.

Grafica:

RECURSOS
BIEN DEFINIDOS

Cliente

STATELESS

Endpoint/
Recurso

Operaciones
HTTP

Respuesta con
Datos + Código
de estado

VERSIONADO

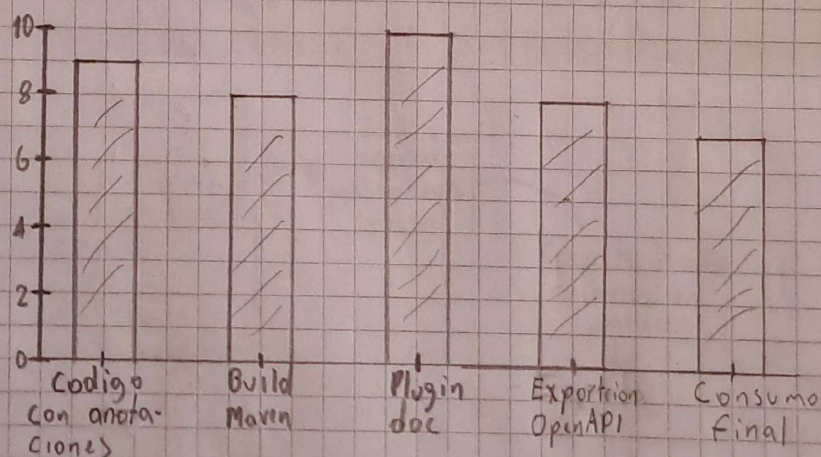
DOCUMENTACION
CLARA

Reflexión: Más que un manual técnico, es un manifiesto de buenas prácticas. Nos recuerda que una API bien diseñada es un contrato de confianza entre sistemas, y que la simplicidad y consistencia son virtudes que perduran.

Artículo 5: Biblioteca Maven para la generación automatizada de documentación de APIs REST desarrolladas en Java y Spring Boot (2022)

Este trabajo presenta una librería Maven diseñada para automatizar la creación de documentación de APIs REST en proyectos desarrollados con Java y Spring Boot. La propuesta busca reducir el tiempo y el esfuerzo necesarios para mantener la documentación actualizada integrándose de forma directa con el ciclo de compilación. El artículo describe cómo la herramienta extrae información de anotaciones y definiciones del código, generando documentación en formato estándar (por ejemplo, OpenAPI/Swagger). Se muestran ejemplos prácticos de uso, pruebas en proyectos reales y métricas de mejora en productividad. Concluye que esta solución es útil para equipos de desarrollo que requieren documentación consistente y alineada con el estado actual del código.

Gráfica:

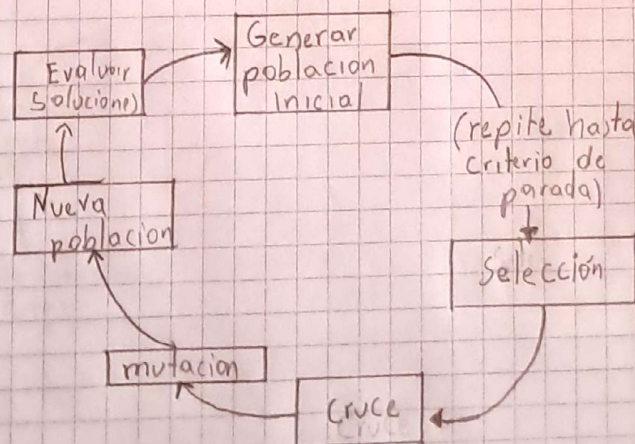


Reflexión: Automatizar la documentación no es solo eficiencia; es garantizar que el conocimiento técnico sea accesible y actualizado. Este artículo subraya que la documentación viva es parte esencial del ciclo de desarrollo.

Artículo 9: Implementación de un generador inteligente de horarios utilizando algoritmo genético en la Universidad de Ciencias y Humanidades (2017)

Este estudio desarrolla un generador inteligente de horarios que aplica algoritmo genético para resolver la compleja tarea de asignar clases, aulas y profesores en una universidad. El enfoque combina técnicas evolutivas con reglas y restricciones propias de la institución, buscando minimizar conflictos y optimizar el uso de recursos. Se detalla el modelado del problema, la representación de las soluciones como cromosomas, y la configuración de operadores de cruce y mutación. El sistema fue probado con datos reales de la Universidad de Ciencias y Humanidades, mostrando una alta tasa de satisfacción de las restricciones y una reducción significativa del tiempo requerido para elaborar horarios respecto al método manual. El artículo sugiere como mejora futura la integración de una interfaz gráfica más intuitiva.

Gráfico:

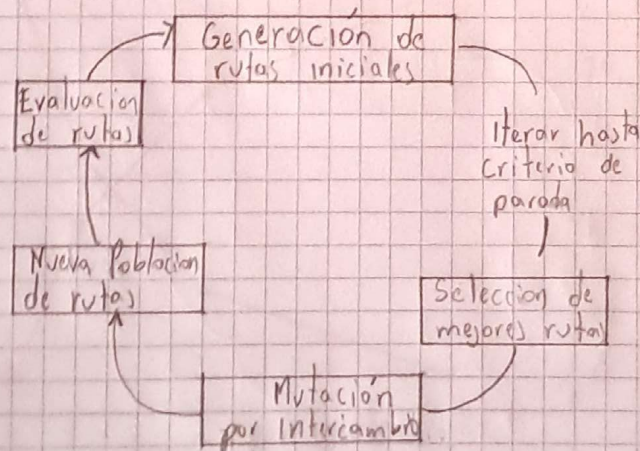


Reflexión: Aquí la inteligencia artificial se pone al servicio de la gestión académica. La reflexión es que la optimización no es un lujo, si no una necesidad para instituciones que buscan adaptarse a la complejidad creciente.

Artículo 10: Generación de horarios mediante algoritmos genéticos (2023)

Este estudio aborda la creación de un sistema para generar horarios académicos optimizados usando algoritmos genéticos, buscando maximizar la eficiencia y minimizar conflictos. El problema se modela como uno de optimización con múltiples restricciones: disponibilidad de docentes, aulas, duración clases, y asignaturas obligatorias. Se describe cómo las soluciones se representan como cromosomas y cómo se aplican operadores de cruce y mutación para explorar el espacio de posibles horarios. Las pruebas realizadas con datos reales mostraron mejoras significativas en la asignación de recursos, reduciendo horas libres y evitando choques de horario. El artículo concluye que esta técnica puede adaptarse a distintos contextos educativos y que, con ajuste en los parámetros del algoritmo, es posible lograr una planificación más flexible y personalizada.

Grafica:

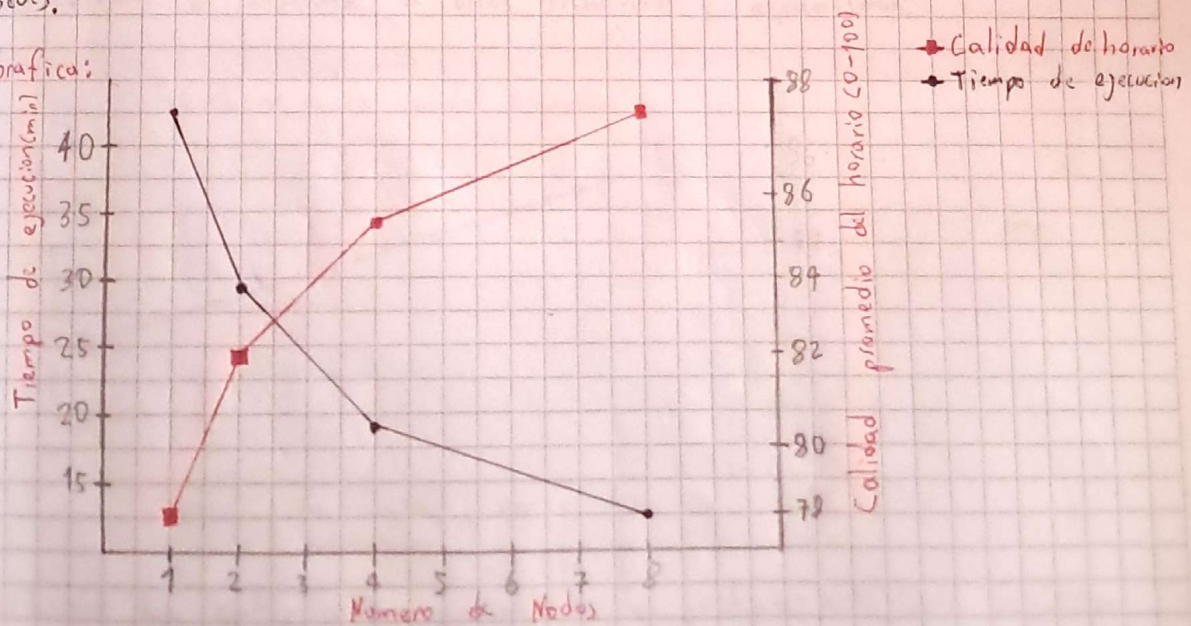


Reflexión: Este trabajo reafirma que los algoritmos genéticos siguen vigentes y adaptables. La lección es que la evolución natural sigue inspirando soluciones a problemas que, aunque modernos, comparten la misma esencia: encontrar el mejor ajuste posible.

Artículo 11: Programación distribuida de horarios en Java utilizando técnicas de algoritmos genéticos (2005)

Este trabajo propone un enfoque de programación distribuida para generar horarios académicos, usando Java como lenguaje de implementación y algoritmos genéticos para la optimización. El sistema divide el problema en subproblemas distribuidos entre varios nodos, lo que permite aprovechar el paralelismo y reducir tiempos de cálculo. Se detallan las estructuras de datos empleadas, las funciones de evaluación y los mecanismos de comunicación entre nodos. Las pruebas evidencian que la distribución de la carga de trabajo no solo acelera el proceso, sino que también permite manejar instancias de mayor complejidad sin sacrificar la calidad de las soluciones. El autor sugiere que este modelo podría escalarse para sistemas de planificación de mayor tamaño, como universidades con múltiples sedes.

Gráfica:

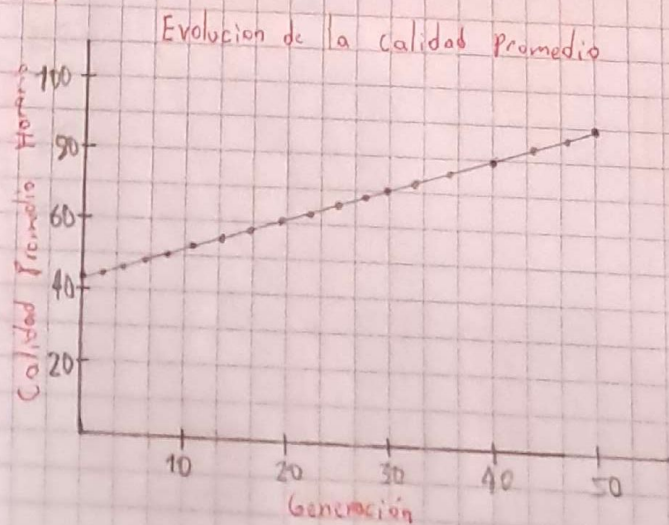


Reflexión: un enfoque pionero que combina optimización y computación distribuida. Nos recuerda que la escalabilidad no es solo manejar más datos, sino distribuir inteligentemente el esfuerzo de cálculo.

Artículo 12: Propuesta de desarrollo de una aplicación web para la asignación de horarios y salas basado en algoritmos genéticos para el juzgado de control, Ejecutorio Penal del Estado de Guerrero (2021)

Este artículo presenta el diseño de una aplicación web para recomendar horarios y asignación de salas en un contexto judicial, aplicando algoritmos genéticos como motor de optimización. Se parte de las necesidades específicas de un juzgado penal, como la disponibilidad de jueces, salas y equipos, y el cumplimiento de plazos procesales. El sistema implementa un modelo que evalúa las posibles asignaciones y selecciona las que mejor cumplen con las restricciones y objetivos. Se incluyen una descripción de la arquitectura de la aplicación, la base de datos y la interfaz de usuario. Los resultados de las pruebas piloto indican mejoras en la utilización de recursos y reducción en los tiempos muertos. El artículo también plantea la viabilidad de extender el sistema a otros tribunales y organismos.

Gráfica

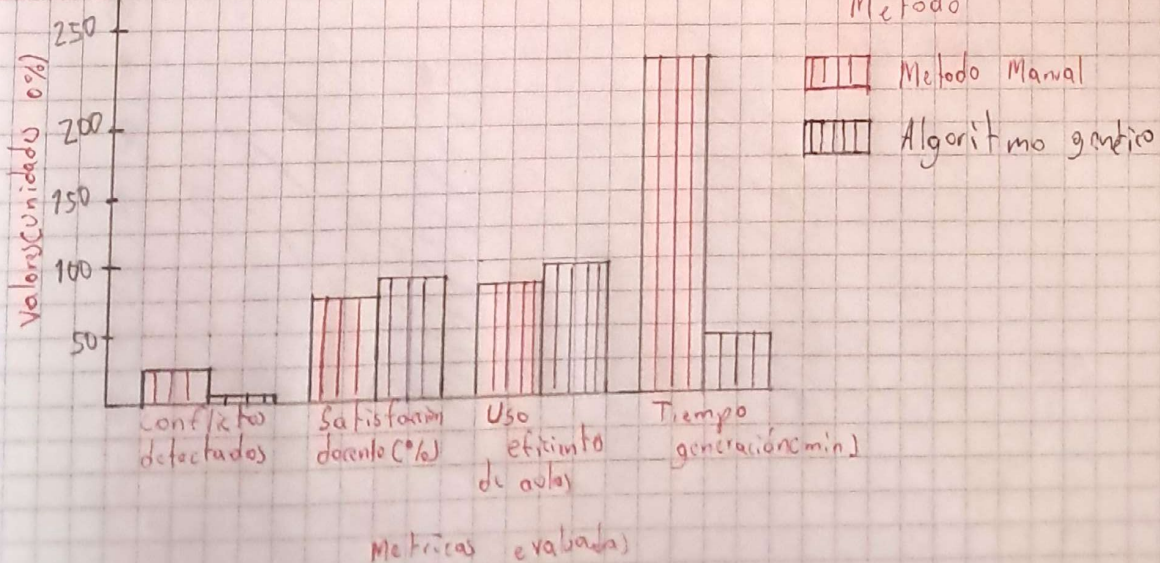


Reflexión: La optimización aplicada a la justicia: este caso demuestra que la tecnología puede agilizar procesos críticos, liberando recursos humanos para tareas de mayor valor.

Artículo 13: Influencia de los algoritmos genéticos en la generación de horarios en la Unidad Educativa Ana Luisa Leoro (2019)

Este trabajo analiza el impacto de la aplicación de algoritmos genéticos en la creación de horarios para una institución educativa específica. Parte de un diagnóstico de los problemas que enfrentaba la escuela: conflictos de horarios, baja eficiencia en la asignación de recursos y exceso de tiempo en la planificación manual. El modelo propuesto representa cada posible horario como un cromosoma y aplica operadores de cruce y mutación para encontrar soluciones cada vez más eficientes. Los resultados muestran una reducción significativa de los conflictos y una optimización del uso de aulas y docentes. El estudio concluye que los algoritmos genéticos no solo mejoran la calidad de los horarios, sino que también liberan tiempo para que el personal se enfoque en otras tareas administrativas.

Grafico:

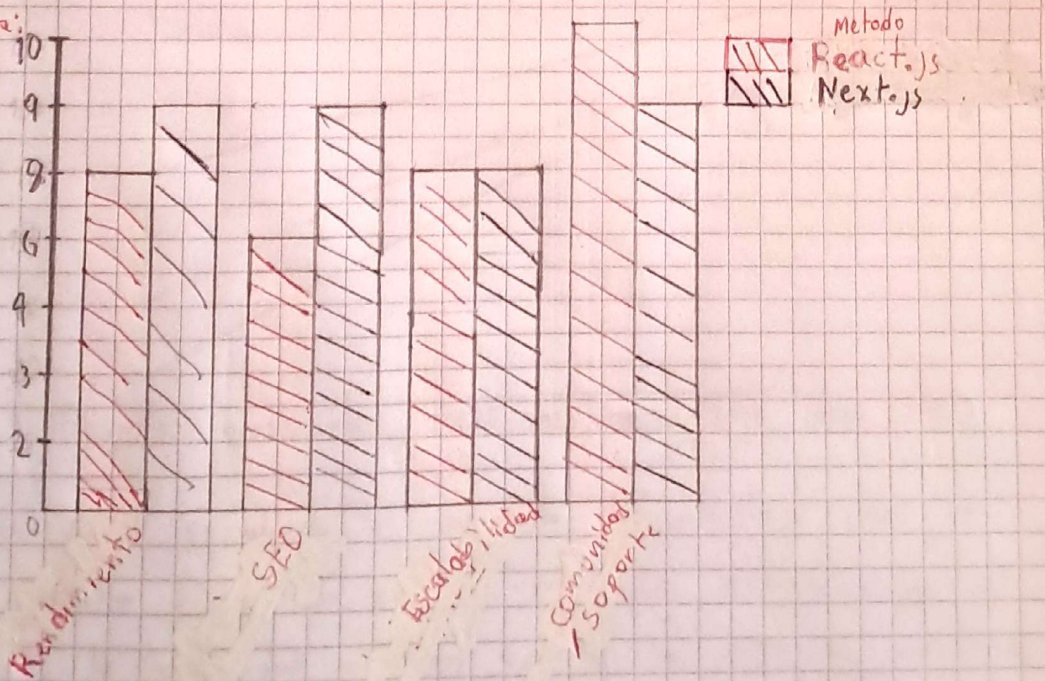


Reflexión: Más que un experimento técnico, es un estudio de impacto. La reflexión es que medir la influencia de una tecnología es tan importante como implementarla.

Artículo 24: Modern Front End Web Architectures with React.js and Next.js (2023)

Este artículo examina las arquitecturas modernas de desarrollo frontend utilizando React.js y Next.js, enfocándose en cómo ambas tecnologías se complementan para crear aplicaciones web rápidas, escalables y optimizadas para SEO. React.js se presenta como una librería flexible para construir interfaces de usuario dinámicas, mientras que Next.js añade características como renderizado del lado del servidor (SSR), generación de sitios estáticos (SSG) y enrutamiento simplificado. El autor muestra ejemplos prácticos, patrones de arquitectura recomendados y beneficios en rendimiento y experiencia de usuario. También se abordan retos como la gestión del estado global y la optimización de recursos, proponiendo soluciones basadas en librerías y buenas prácticas ampliamente adoptadas en la industria.

Gráfica:



Reflexión: Este artículo muestra cómo la arquitectura moderna del frontend combina rendimiento, SEO y experiencia de usuario.

Lección: La frontera entre backend y frontend es cada vez más difusa, y esto abre nuevas posibilidades.

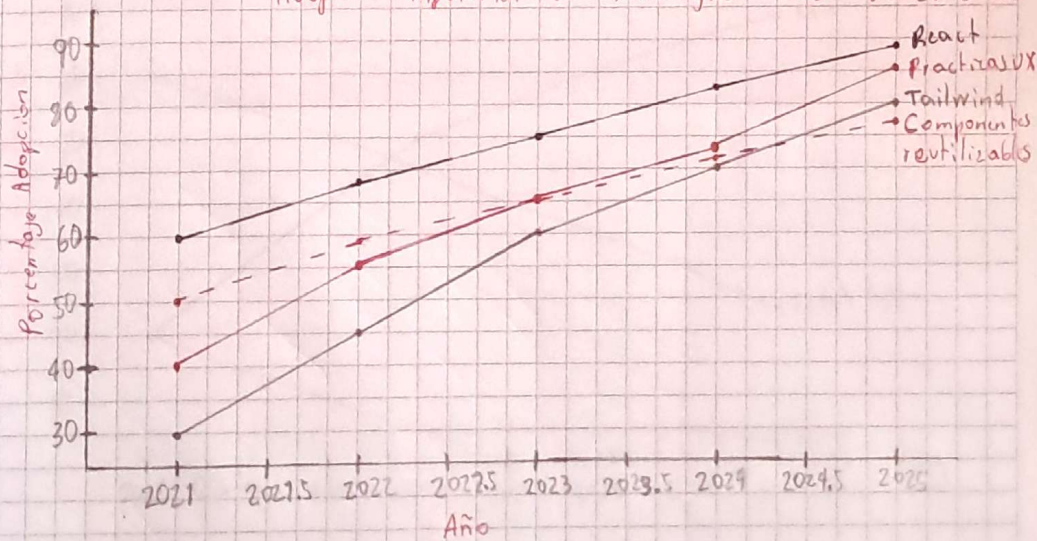
Artículo: User Interface Development of a Modern Web Application (2021)

Este estudio documenta el desarrollo de una interfaz de usuario moderna usando Tailwind CSS junto con React. Se describe cómo Tailwind facilita la creación rápida de componentes personalizados mediante utilidades predefinidas, reduciendo la necesidad de escribir CSS desde cero.

El proyecto presentado integra React para la construcción de una interfaz interactivo y reactiva, mientras que Tailwind asegura consistencia visual y diseño responsivo. Se explican decisiones de arquitectura, manejo del estado y optimización de la experiencia del usuario. El artículo concluye que la combinación de Tailwind y React ofrece un flujo de trabajo más eficiente, reduce la complejidad del código CSS y acelera la entrega de interfaces atractivas y funcionales.

Gráficas:

Adopción Hipotética de Tecnologías Web (2021-2025)

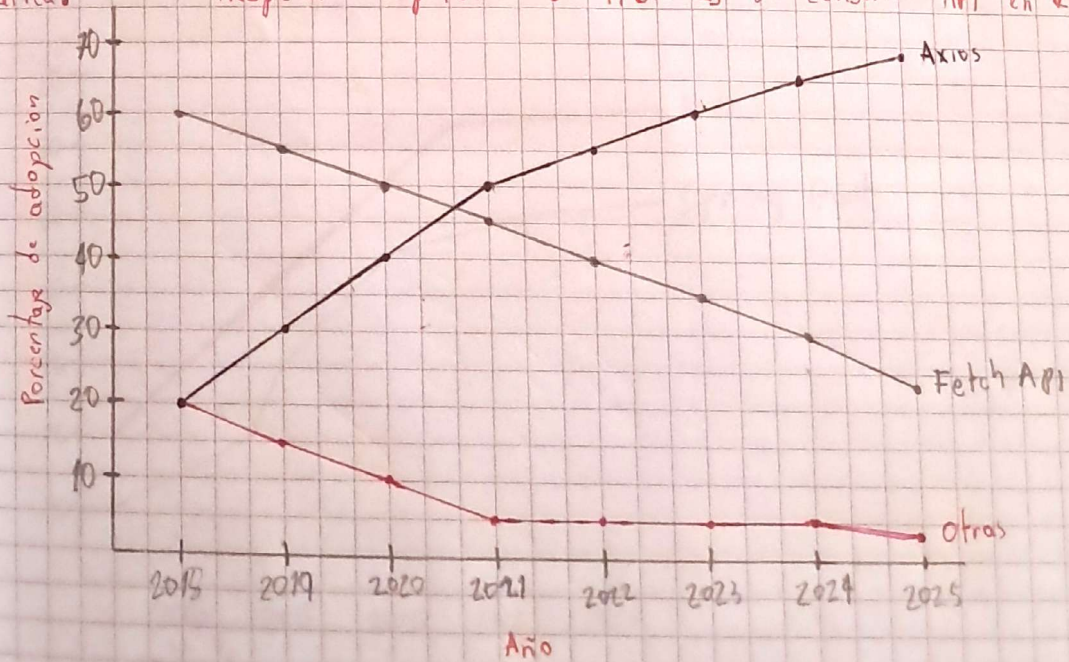


Reflexión: Aquí se aborda el desarrollo de interfaces como un proceso iterativo y centrado en el usuario. La reflexión es que la estética y la usabilidad no son opuestas: se potencian mutuamente.

Estudio de Implementación API en ReactJS (2020)

Este trabajo documenta el proceso de integrar y consumir APIs dentro de aplicaciones creadas con ReactJS, utilizando la librería Axios para gestionar solicitudes HTTP. El autor explica cómo estructurar componentes para manejar llamadas asíncronas, gestionar el estado de la aplicación y tratar posibles errores. Se incluyen para autenticación. También se abordan buenas prácticas como la separación de la lógica de red en servicios independientes, la gestión del estado global y la optimización de la experiencia del usuario mediante indicadores de carga y manejo de errores. El estudio concluye que el uso de Axios con React simplifica la integración con APIs y mejora la mantenibilidad del código.

Grafica: Adopción hipotética de librerías de consumo API en ReactJS



Reflexión: Un recordatorio de que la integración entre frontend y backend es un arte de precisión. La lección: una API mal consumida puede arruinar la mejor interfaz.

Artículo de desarrollo backend para aplicaciones web, servicios Web REST, análisis de Spring Boot (2016)

Este artículo compara dos tecnologías para el desarrollo backend: Node.js y Spring Boot. Presenta sus arquitecturas, el manejo de solicitudes REST y el rendimiento en distintos escenarios. Se evalúan criterios como escalabilidad, Facilidad de configuración, ecosistema de librerías y curva de aprendizaje. Node.js destaca por su naturaleza asíncrona y gran comunidad de módulos, mientras que Spring Boot ofrece robustez, integración con el ecosistema Java y herramientas avanzadas para proyectos complejos. El autor analiza casos de uso y propone criterios para elegir la tecnología adecuada según el tipo y las necesidades del proyecto. Como conclusión, sugiere que ambas opciones son válidas, pero su idoneidad depende del contexto.

Gráfico:

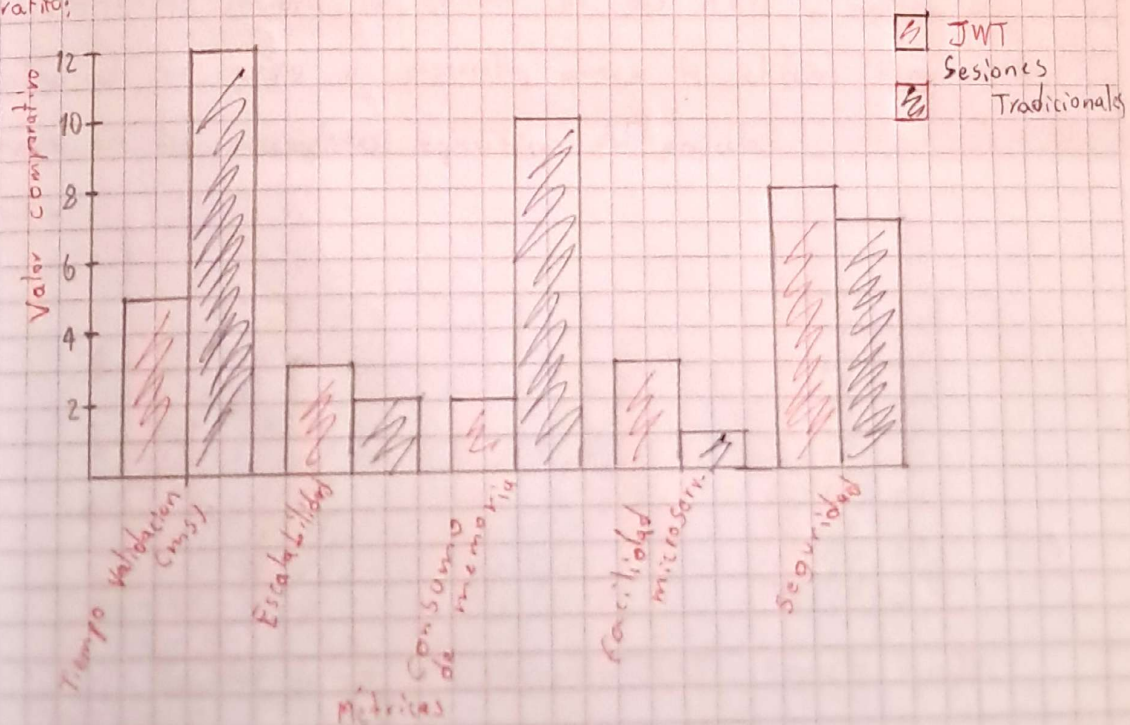


Reflexión: Más que una comparación técnica, es una reflexión sobre elegir la herramienta adecuada para el contexto. No hay ganadores absolutos, solo soluciones más o menos alineadas con las necesidades.

Artículo 18. JWT Based Authentication (2020)

Este estudio describe el uso de JSON Web Tokens (JWT) como mecanismo de autenticación y autorización en aplicaciones web y móviles. Explica la estructura de un token (header, payload, signature) y cómo se utiliza para validar la identidad del usuario sin necesidad de mantener sesiones en el servidor. Se detalla el flujo de autenticación: generación del token, envío al cliente y validación en solicitudes posteriores. También se abordan aspectos de seguridad como la expiración de tokens, su almacenamiento seguro en el cliente y estrategias para evitar ataques de tipo replay o robo de credenciales. El artículo concluye que JWT ofrece una solución ligera y escalable, especialmente útil en arquitecturas distribuidas y APIs REST.

Gráfico:

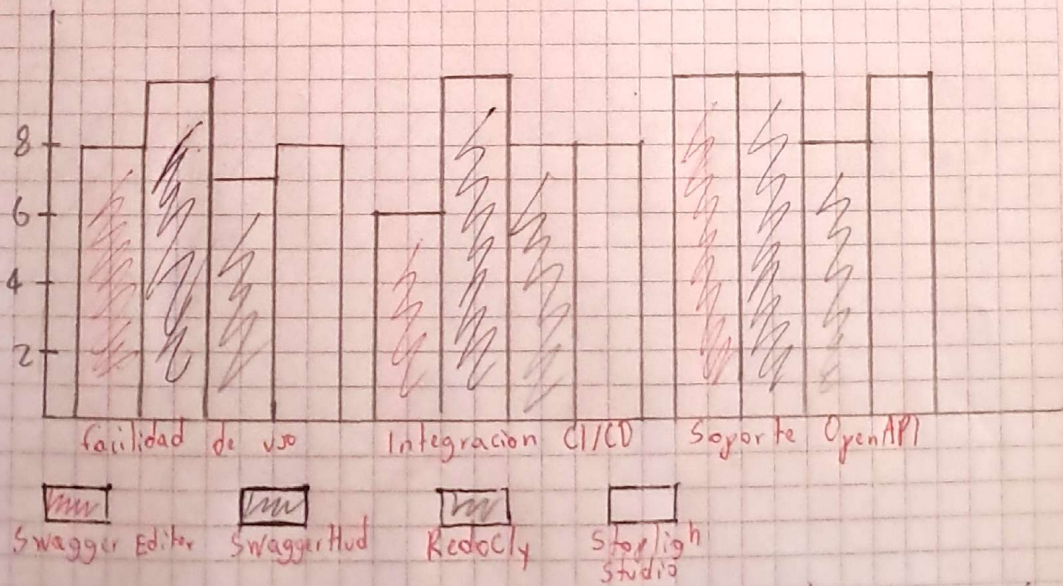


Reflexión: La autenticación basada en JWT es un ejemplo de cómo la seguridad puede ser ligera y escalable. La reflexión protege datos no tiene por qué sacrificar la experiencia del usuario.

Análisis de Herramientas para REST Contract Specification en Swagger y OpenAPI (2020)

Este trabajo examina diferentes herramientas para la especificación de contratos REST usando los estándares Swagger y OpenAPI. Comienza explicando la relevancia de los contratos en APIs, ya que garantizan una comunicación clara entre el cliente y el servidor, reduciendo errores de integración. El artículo compara varias soluciones que facilitan la generación, validación y mantenimiento de contratos, evaluando criterios como compatibilidad con distintos lenguajes, soporte de documentación automática y facilidad de uso. Se describen casos de aplicación y se analizan ventajas como la generación de código y la detección temprana de inconsistencias. Concluye que el uso de herramientas estandarizadas mejora la calidad de las APIs y agiliza el trabajo de los equipos de desarrollo, aunque la elección final debe basarse en las necesidades específicas del proyecto.

Grafica:



Reflexión: Este análisis recuerda que estandarizar la comunicación entre servicios es clave para la interoperabilidad. La lección: un contrato claro evita malentendidos costosos.

Artículo 20: Estudio comparativo de los frameworks del desarrollo móvil nativo "Flutter" y "React Native" (2021)

Este estudio realiza una comparación técnica entre Flutter y React Native para el desarrollo de aplicaciones móviles nativas, se analizan aspectos como arquitectura, lenguaje de programación (Dart para Flutter, JavaScript para React Native), rendimiento, personalización de interfaces y ecosistema de librerías. El autor presenta pruebas de rendimiento y calidad de la interfaz, así como la curva de aprendizaje de cada framework. Flutter destaca por su motor gráfico propio y alta consistencia visual en múltiples plataformas, mientras que React Native se beneficia de una comunidad madura y un amplio soporte de librerías. El artículo concluye que ambos frameworks son viables, pero la decisión debe considerar factores como la experiencia del equipo, el tipo de proyecto y las necesidades de mantenimiento a largo plazo.

Gráfica:

